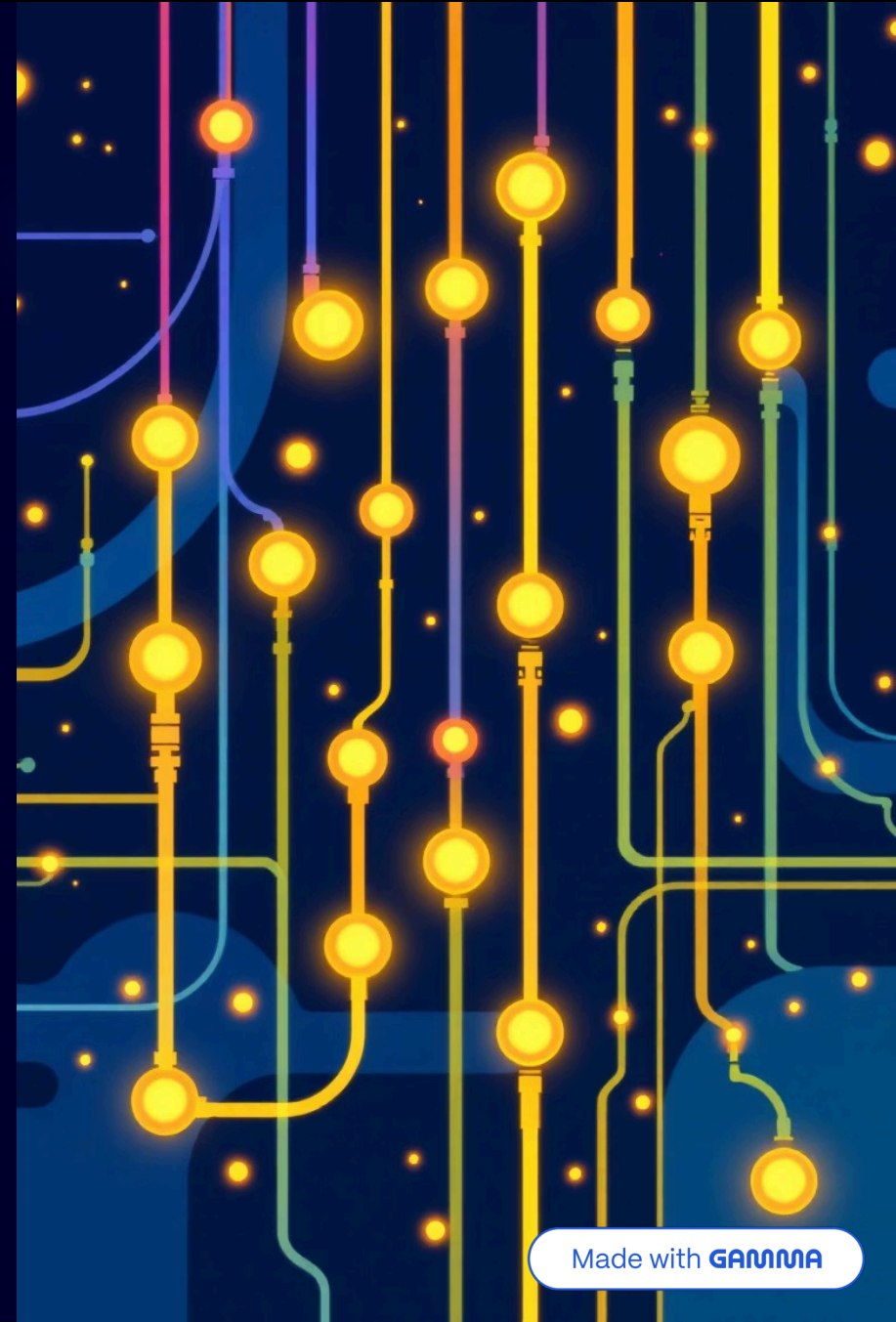


OSI Layer Emulator

Interactive Encapsulation & Signal Visualization

A web-based simulation demonstrating the complete data encapsulation process through all seven layers of the OSI model, built with HTML, CSS, and JavaScript



Project Objective

Why the OSI Model is Challenging

Network students often struggle with abstract concepts in OSI theory. Textbooks describe each layer in isolation, but real networking requires understanding how data transforms as it passes through multiple layers simultaneously. The gap between theoretical knowledge and practical implementation remains significant.

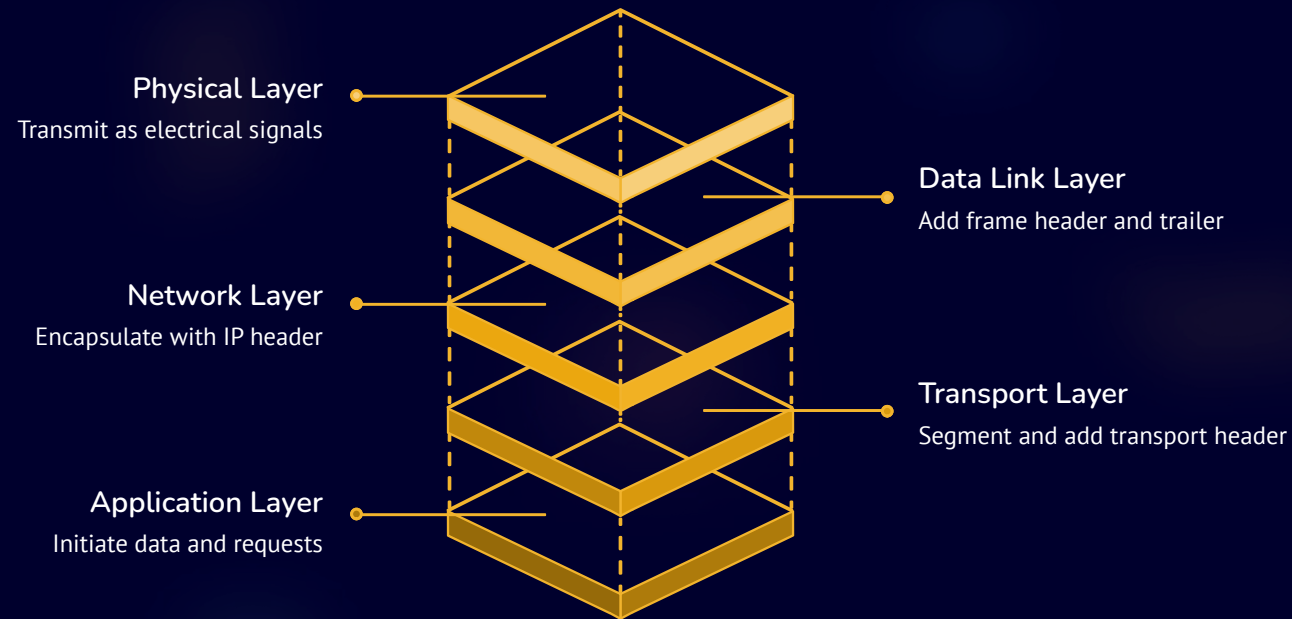
Educational Impact: Provides immediate visual feedback on networking concepts, reduces cognitive load through progressive visualization, and allows experimentation without network hardware.

How This Emulator Helps

This project bridges theory and practice by visualizing the complete data flow from application input to physical transmission. Students can manipulate parameters at each layer, observe encapsulation in real-time, and see how headers are added sequentially. The simulation demonstrates both logical data structures and physical signal representations.

Overall Data Flow in the Emulator

Data travels downward through each OSI layer, with each layer adding its own header information—a process called **encapsulation**. The completed frame is then transmitted as electrical, optical, or radio signals over the physical medium.



01	02	03
Application Layer User data created and submitted	Presentation Layer Data encoded and optionally encrypted	Session Layer Session established with unique ID
04	05	06
Transport Layer Data segmented with port information	Network Layer IP addressing and routing headers added	Data Link Layer Frame constructed with MAC addresses
07		
Physical Layer Bits encoded as physical signals		

📌 **Logical vs. Physical:** Higher layers work with structured data and headers, while the Physical Layer deals exclusively with signal transmission techniques like encoding and modulation.

Upper Layers: Application, Presentation, and Session

Application Layer (Layer 7)

Function: User data input and network service interface

The Application Layer represents the actual data generated by user applications—HTTP requests, file transfers, email messages, or database queries. In the emulator, this is where users input their test data, which can be plain text or simulated application data.

OSI Definition: Provides network services directly to user applications and serves as the interface between software and network functionality.

Presentation Layer (Layer 6)

Function: Data translation, encryption, and compression

This layer transforms data into a format suitable for network transmission. The emulator demonstrates character encoding (ASCII, UTF-8), optional encryption using simulated cryptographic algorithms, and data compression. The encoded data remains logically intact but appears transformed to external observers.

OSI Definition: Translates data between application and network formats, ensuring data is syntactically and semantically correct.

Session Layer (Layer 5)

Function: Session establishment and management

The Session Layer creates, maintains, and terminates communication sessions between applications. The emulator generates unique session identifiers, manages session state, and handles synchronization points. This layer ensures that multiple application processes can communicate simultaneously without interference.

OSI Definition: Establishes, manages, and terminates connections between applications, providing mechanisms for dialogue control and synchronization.

Transport Layer: TCP and UDP Protocols

Core Functions

The Transport Layer segments data from upper layers and adds port numbers for application multiplexing. Users can select between TCP (connection-oriented, reliable) and UDP (connectionless, fast) protocols based on application requirements.

TCP Header Structure

- **Source Port:** Sender's application port (16 bits)
- **Destination Port:** Receiver's application port (16 bits)
- **Sequence Number:** Byte position in data stream (32 bits)
- **Acknowledgment Number:** Next expected byte (32 bits)
- **Flags:** SYN, ACK, FIN, RST control bits
- **Window Size:** Flow control buffer space

UDP Header Structure

- Source Port (16 bits)
- Destination Port (16 bits)
- Length (16 bits)
- Checksum (16 bits)

TCP vs UDP Selection

TCP provides reliability through acknowledgments and retransmissions. UDP offers minimal overhead for real-time applications. The emulator allows users to compare packet sizes and transmission characteristics.

Segmentation Process

Large data streams are divided into manageable segments. Each segment receives a sequence number for reassembly. The emulator visualizes segment boundaries and shows how data is progressively wrapped with transport headers.

Network Layer: IP Addressing and Routing

The Network Layer adds logical addressing information that enables routing across multiple network segments. Each packet receives source and destination IP addresses, along with fields controlling packet lifetime and protocol identification.

1

IP Header Construction

Version (IPv4/IPv6), Time-to-Live (TTL) for loop prevention, Protocol field identifying upper-layer protocol (TCP=6, UDP=17), and 32-bit source and destination addresses are added to each packet.

2

Packetization Process

Transport segments become IP packet payloads. The Network Layer may further fragment packets to fit underlying link MTU (Maximum Transmission Unit) limitations. Each fragment receives sequencing information for reassembly.

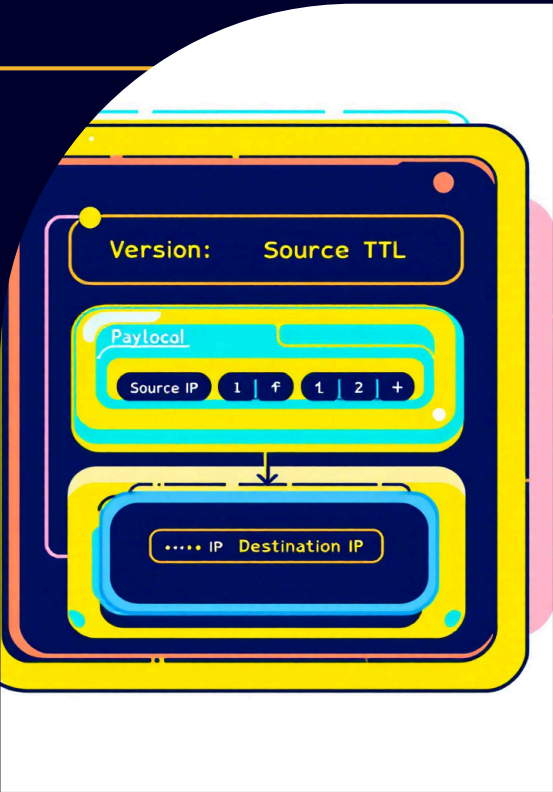
3

Router Forwarding

Routers examine destination IP addresses and consult routing tables to determine next-hop interfaces. The emulator simulates routing decisions based on longest-prefix matching algorithms and demonstrates how packets traverse multiple hops.

IP Header

Contains Version, TTL, and Protocol fields for routing control.



Addresses

Source IP and Destination IP identify packet endpoints across networks.



Key Insight: The Network Layer treats packets as opaque data units. It doesn't interpret transport or application content—only logical addressing for inter-network delivery.

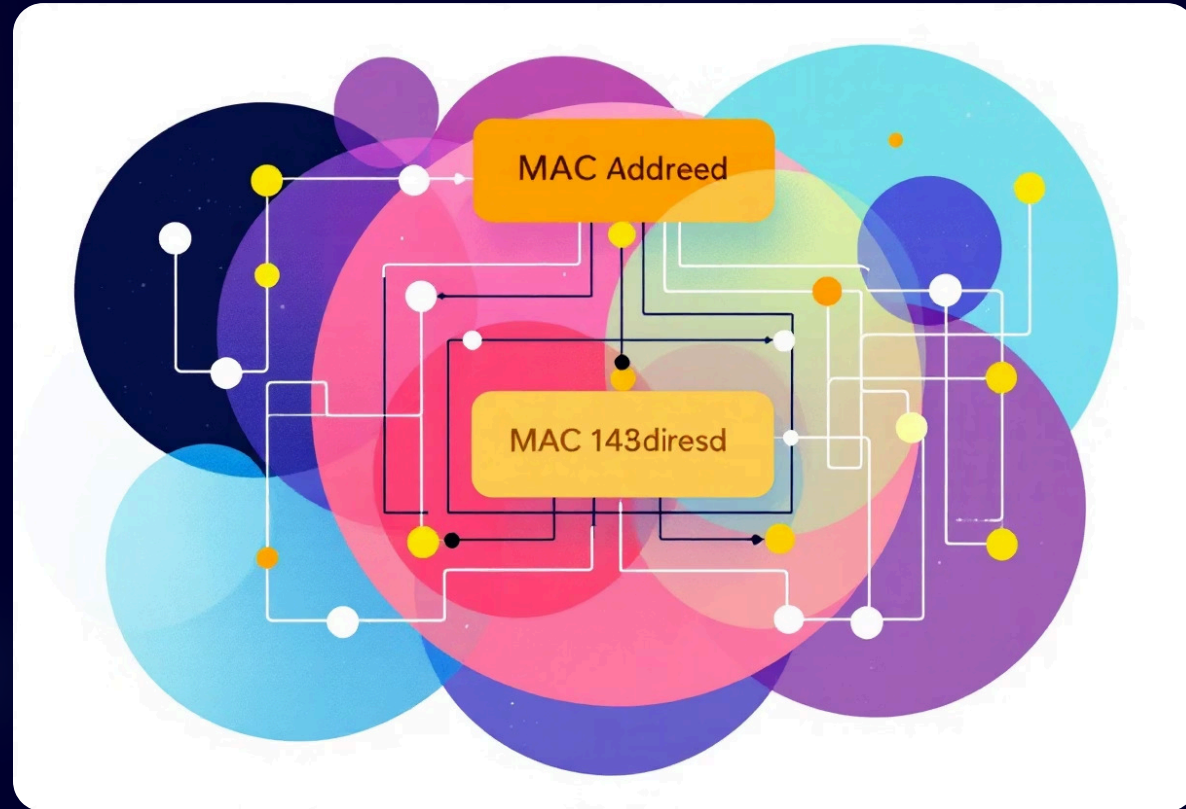
Data Link Layer: Framing and Error Detection

Frame Construction

The Data Link Layer encapsulates IP packets into frames suitable for transmission over physical media. Each frame includes header fields with MAC addresses for local delivery within the same broadcast domain.

Frame Structure:

- Frame Header: Source and destination MAC addresses (48 bits each)
- Payload Area: Encapsulated IP packet (variable length)
- Frame Trailer: Frame Check Sequence (FCS) for error detection



MAC Addressing

Hardware addresses uniquely identify network interfaces within a local network segment. The emulator shows address resolution using simulated ARP (Address Resolution Protocol) requests.



Parity Check

Simple error detection adding a parity bit to ensure even or odd bit counts. Limited effectiveness but minimal overhead. The emulator demonstrates single-bit error detection capability.



Cyclic Redundancy Check (CRC)

Polynomial-based hash function generating FCS values. Highly effective at detecting burst errors. The emulator implements CRC-32 calculation and shows detection of corrupted frames during transmission.

Important: Error detection mechanisms identify corrupted frames but do not correct them—detection triggers retransmission requests at higher layers.

Physical Layer: Encoding and Modulation

The Physical Layer converts frame bits into physical signals suitable for transmission over cables, fiber optics, or wireless channels. This layer works exclusively with **signals**, not data structures or logical meanings.



Line Encoding Techniques

- NRZ (Non-Return to Zero):** High voltage = 1, low voltage = 0. Simple but lacks synchronization.
- Manchester Encoding:** Each bit period splits into two halves. Transition from low-to-high represents 1, high-to-low represents 0. Self-synchronizing with clock embedded.
- Differential Manchester:** Always transitions at bit start. Data determined by whether transition occurs mid-bit. More robust synchronization.



Modulation Techniques

- ASK (Amplitude Shift Keying):** Varies signal amplitude to represent 0 and 1. Susceptible to noise.
- FSK (Frequency Shift Keying):** Changes frequency between two values. More noise-resistant than ASK.
- PSK (Phase Shift Keying):** Shifts signal phase (0° vs 180°). High noise immunity and spectral efficiency.



Signal Visualization

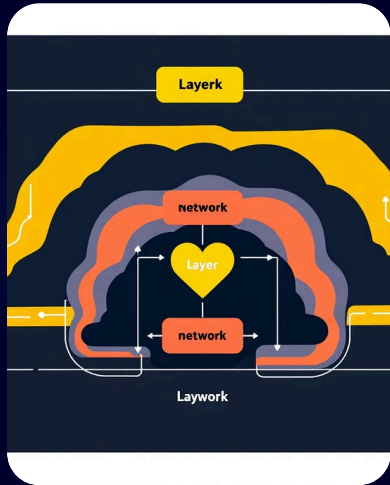
The emulator renders real-time waveform graphs showing how bit sequences transform into electrical or optical signals. Users can compare encoding efficiency and observe timing relationships.

Medium Characteristics

Physical Layer implementation varies dramatically by transmission medium—copper wire, fiber optic cable, or radio frequency. Each medium imposes different constraints on encoding and modulation choices.

Visualizations in the Emulator

The emulator provides four distinct visualization modes, each emphasizing different aspects of the encapsulation process. These views help students connect abstract OSI theory with concrete data transformations.



Encapsulation View

Shows progressive wrapping of data as it passes through each layer. Header fields appear sequentially with color-coded highlighting. Students can expand each layer to inspect individual header values and understand how encapsulation builds nested structures.



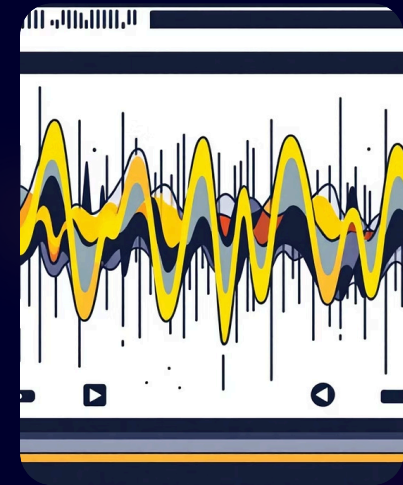
Segmentation/Framing View

Visualizes how data divides into segments at the Transport Layer, packets at the Network Layer, and frames at the Data Link Layer. Boundary markers show exact byte positions, and size calculations demonstrate overhead introduced by each encapsulation step.



Packet Flow View

Traces complete lifecycle of data from application input through physical transmission and (future) receiver-side decapsulation. Timeline animations show sequence numbers, acknowledgment patterns, and retransmission events for TCP connections.



Signal Graph

Real-time oscilloscope-style visualization of Physical Layer encoding. Users can toggle between NRZ, Manchester, and Differential Manchester to observe waveform differences. Amplitude, frequency, and phase modulation displays show how bits map to analog signals.

 **Educational Value:** Multiple visualization perspectives reinforce conceptual understanding by showing the same data transformation from different angles—structural, temporal, and physical.

Conclusion and Learning Outcomes

Connecting Theory to Practice

The OSI Layer Emulator bridges abstract networking concepts with observable behavior. Students manipulate real parameters and immediately see encapsulation effects, transforming textbook diagrams into interactive learning experiences.

Key Concepts Demonstrated

Complete encapsulation process, header field semantics, protocol differences (TCP vs UDP), IP addressing and routing, MAC addressing and local delivery, error detection mechanisms, and physical signal encoding.

Visualization-Based Learning

Progressive layer-by-layer visualization reduces cognitive load. Multiple representation modes (structural, temporal, waveform) accommodate different learning styles and reinforce conceptual connections.

Future Improvements

Receiver-Side Decapsulation

Current implementation shows transmission from sender perspective. Future version will simulate receiver processing—stripping headers, reassembling segments, and delivering data to application layer.

Error Injection and Recovery

Add simulated packet loss, corruption, and retransmission to demonstrate TCP reliability mechanisms. Students can configure error rates and observe acknowledgment patterns and timeout behaviors.

Educational Impact: This emulator provides networking students with hands-on experience of OSI encapsulation that would otherwise require packet capture tools, network analyzers, and hardware—making advanced concepts accessible through standard web browsers.